O2C Graph Intelligence System

Technical Documentation

SAP Order-to-Cash Dataset — Interactive Graph with Natural Language Query Interface

Live Demo: https://llm-powered-o2c.vercel.app

Backend API: https://llm-powered-o2c-production.up.railway.app

1. Project Overview

This system ingests the SAP Order-to-Cash (O2C) dataset and transforms it into an interactive graph intelligence platform. Users can:

- Visually explore relationships between business entities (orders, deliveries, billing documents, payments, customers, products)
- Query the data in natural language — the system translates the question into SQL, executes it against a live PostgreSQL database, and returns a human-readable answer
- Trace complete O2C flows — from a Sales Order through Delivery → Billing → Journal Entry → Payment
- Detect broken flows — orders without deliveries, billing without payment, cancelled documents

The three core components are:

1. Graph Construction & Visualization — Entities modeled as nodes, FK relationships as edges, rendered with React Flow
2. Conversational Query Interface — Natural language → SQL via Groq LLM → executed on PostgreSQL → formatted answer
3. Guardrail System — Multi-layer protection ensuring only O2C-relevant queries are answered

2. Tech Stack

Layer	Technology	Rationale
Frontend	React 18 + TypeScript + Vite	Type-safe, fast HMR, component-driven UI
Graph Visualization	React Flow + Dagre	Purpose-built for node/edge graphs; Dagre computes automatic LR layout
Backend	Go + Fiber	High performance, low latency, strong typing for data pipelines
Database	PostgreSQL (Supabase)	Native relational model fits O2C FK chains
LLM	Groq API — Llama 3.3 70B	Free tier (14,400 req/day), ~300ms response, superior SQL generation
Deployment	Vercel + Railway + Supabase	Free usage, purpose-optimized for each layer

3. Architecture Decisions

3.1 Why PostgreSQL Over a Graph Database

The O2C dataset is fundamentally relational — every relationship follows a well-defined foreign key chain:

```
sales_order → delivery → billing → journal_entry → payment
```

A graph database (Neo4j, ArangoDB) would add operational complexity without benefit because:

- All relationships are known and fixed, not discovered dynamically
- SQL JOIN queries are more readable and debuggable than Cypher
- PostgreSQL handles FK traversal efficiently with indexes
- Supabase provides managed PostgreSQL with a free tier

The graph in this system is a logical representation built from relational FK relationships — not a separate storage layer. Node and edge data is derived at query time from existing table structure.

3.2 Why Go + Fiber for the Backend

- Compiled binary = fast cold starts on Railway
- Static typing catches data pipeline bugs at compile time
- Strong stdlib — minimal third-party dependencies
- Fiber's Express-like API is familiar and minimal

- goroutines handle concurrent DB queries efficiently

3.3 Why Groq Over Gemini or OpenAI

Groq was selected after testing multiple free-tier LLM providers:

Provider	Issue / Advantage
Gemini (free)	Strict quota limits — frequent failures during development
OpenAI	No meaningful free tier
Groq (Llama 3.3 70B)	14,400 req/day free, ~300ms latency, accurate complex SQL generation

3.4 Clean Architecture

The backend follows Clean Architecture with a strict separation of concerns:

```
backend/
├── cmd/main.go                        ← Entry point only
├── internal/
│   ├── core/domain/                   ← Pure business entities (Node, Edge,
ChatRequest)
│   ├── core/usecases/                 ← Business logic (SQL generation, LLM
calls)
│   ├── adapter/db/                    ← PostgreSQL repository implementations
│   ├── adapter/http/handlers/         ← Fiber HTTP handlers
│   └── infra/                         ← Migration and ingestion utilities
├── config/                            ← Environment config
└── schema.sql                         ← DDL for all 19 tables
```

This separation ensures business logic has zero dependency on the HTTP framework or database driver — each layer is independently testable and replaceable.

4. Graph Modeling

4.1 Node Types

Node Type	Source Table	Key Label Field	UI Color
SalesOrder	sales_order_headers	sales_order	Blue
Delivery	outbound_delivery_headers	delivery_document	Purple
BillingDocument	billing_document_headers	billing_document	Amber

Node Type	Source Table	Key Label Field	UI Color
Payment	payments_ar	accounting_document	Green
Customer	business_partners	business_partner_name	Teal
Product	products + product_descriptions	product_description	Orange
Plant	plants	plant_name	Gray

4.2 Edge Relationships

Edge Label	Source → Target	Join Condition
placed_by	SalesOrder → Customer	sold_to_party = business_partner
delivers	SalesOrder → Delivery	outbound_delivery_items.reference_sd_document = sales_order
bills	Delivery → BillingDocument	billing_document_items.reference_sd_document = delivery_document
journalized	BillingDocument → JournalEntry	journal_entry_items_ar.reference_document = billing_document
paid_by	JournalEntry → Payment	payments_ar.accounting_document = clearing_accounting_document
contains	SalesOrder → Product	sales_order_items.material = product

4.3 Graph Layout

Dagre.js computes automatic left-to-right layout following the O2C business flow:

```
Customer → SalesOrder → Delivery → BillingDocument → JournalEntry → Payment
```

This makes the business process flow visually obvious without any manual node positioning.

5. LLM Prompting Strategy

5.1 Two-Stage LLM Architecture

The chat system makes two separate Groq API calls per user query:

Stage 1 — SQL Generation (GenerateSQL)

- System prompt contains full schema (all 19 tables + columns + FK relationships)
- 9 working SQL examples embedded in the prompt covering all query patterns

- Temperature set to 0.1 for deterministic, consistent SQL output
- Model: llama-3.3-70b-versatile — 70B parameters required for accurate complex multi-table JOINs
- User message contains only the raw natural language question (never mixed into system prompt)

Stage 2 — Answer Formatting (FormatAnswer)

- Separate Groq call with a different system prompt focused on natural language output
- SAP terminology mapping included: accounting_document = "journal entry number"
- Instructs model to be confident and direct — never hedge with "not available" when data exists
- Temperature: 0.1 for consistent formatting

5.2 Key Prompt Engineering Decisions

Separate System Prompt from User Message

Early testing showed that embedding the question inside the system prompt caused the model to interpolate question text directly into SQL WHERE clauses. Proper role separation (system vs user message) eliminates this entirely.

9 Concrete SQL Examples in the Prompt

LLMs learn from examples more reliably than from rules alone. Each example covers a distinct query pattern:

- Simple COUNT with no GROUP BY
- LEFT JOIN null detection (orders without deliveries)
- Multi-table flow trace using LEFT JOINs
- Aggregation with GROUP BY
- Billing vs payment reconciliation

Explicit UNION Prohibition

Without explicit instruction, Llama 3.x defaults to UNION ALL for multi-entity queries. UNION requires identical column counts — which fails when joining tables with different schemas. The prompt states: "NEVER USE UNION OR UNION ALL — use LEFT JOIN instead for multi-table queries."

Temperature 0.1

At default temperature (1.0), the model generates different SQL for the same question on different requests. Temperature 0.1 produces near-deterministic output — critical for a production query system where the same question should always return the same answer.

Full Schema in Every Request

The Groq API is stateless. Every request must include the complete schema context. The schema string (~3KB) is small relative to the 8K context window and ensures the model never hallucinates column names.

6. Guardrail System

6.1 Two-Layer Protection

Layer 1 — Keyword Check (Go code, before any LLM call)

The user query is scanned for O2C domain keywords before any API call is made:

```
order, delivery, billing, invoice, payment, customer, product, material, plant,
sales, shipment, journal, revenue, amount, quantity, document, dispatch,
warehouse, stock, vendor
```

If zero keywords match, the system immediately returns: "This system is designed to answer questions related to the provided dataset only." No Groq API call is made — saving quota and adding zero latency.

Layer 2 — LLM Classification (Groq system prompt)

The system prompt instructs the model: "If the question is unrelated to this SAP O2C dataset, respond with exactly: OFFTOPIC". If Groq returns OFFTOPIC, the handler returns the guardrail message without executing any SQL.

6.2 SQL Sanitization (Post-LLM)

Before executing Groq's output against PostgreSQL, the sanitizeSQL function:

- Strips markdown code fences (```sql ... ```) — LLMs sometimes wrap output
- Removes trailing semicolons — can cause driver-level errors
- Trims whitespace

This prevents execution failures caused by LLM formatting habits.

6.3 Error Handling Strategy

SQL execution errors return HTTP 400 (not 500) with a human-readable message. This prevents exposing internal database errors to the frontend while giving the user actionable feedback to rephrase their question.

6.4 Guardrail Test Cases

Query	Expected Response
What is the capital of France?	Guardrail message
Write me a poem	Guardrail message
Who is Elon Musk?	Guardrail message
How many sales orders are there?	Valid SQL + answer
Trace billing document 90504248	Full O2C flow trace

7. Data Ingestion Pipeline

The dataset consists of 19 folders of JSONL files (~21,000 total rows across all entities).

7.1 Ingestion Order (FK Dependency Sequence)

Step	Table
1	products
2	product_descriptions
3	plants
4	product_plants
5	product_storage_locations
6	business_partners
7	business_partner_addresses
8	customer_company_assignments
9	customer_sales_area_assignments
10	sales_order_headers
11	sales_order_items
12	sales_order_schedule_lines
13	outbound_delivery_headers
14	outbound_delivery_items
15	billing_document_headers
16	billing_document_cancellations
17	billing_document_items

Step	Table
18	journal_entry_items_ar
19	payments_ar

7.2 Design Decisions

- ON CONFLICT DO NOTHING on all inserts — makes the pipeline fully idempotent. The server can restart without re-inserting duplicates.
- bufio.Scanner reads JSONL files line by line — memory efficient for large files.
- camelCase → snake_case mapping — JSON field names are mapped to PostgreSQL column names explicitly, not dynamically. This prevents column mapping bugs at runtime.
- Run once at startup — ingestion checks if tables are populated before running, keeping subsequent server starts fast.

8. Deployment Architecture

```
Vercel (Frontend) → Railway (Backend: Go + Fiber) → Supabase
(PostgreSQL)
                                                  ↘ Groq API (Llama 3.3 70B)
```

Layer	Platform	URL
Frontend	Vercel	https://llm-powered-o2c.vercel.app
Backend	Railway (:8080)	https://llm-powered-o2c-production.up.railway.app
Database	Supabase (PostgreSQL)	19 tables, ~21K rows
LLM	Groq API	Llama 3.3 70B — SQL Generation & Formatting

Security Implementation

- All secrets (DATABASE_URL, GroqAPIKey) are environment variables — never in source code
- Railway and Vercel auto-redeploy on every GitHub push to main
- Supabase direct connection (port 5432) used — not the pooled connection — to avoid PgBouncer prepared statement conflicts with lib/pq

9. API Endpoints

Method	Endpoint	Description	Response
GET	/health	Server health check	{"status":"ok"}
GET	/api/graph	All nodes + edges for graph render	{nodes[], edges[]}
GET	/api/node/:type/:id	Single node details + neighbors	{node, neighbors[], edges[]}
POST	/api/chat	NL query → SQL → answer	{answer, sql, rows[], highlight_ids[]}

The highlight_ids field in the chat response contains document IDs referenced in the answer — the frontend uses these to highlight and pan to the relevant nodes in the graph.

10. Example Queries

Query Type	Example Question	What It Tests
Simple count	How many sales orders are there?	Basic COUNT
Aggregation	Which customer has the highest total order value?	GROUP BY + SUM
Flow trace	Trace the full flow of billing document 90504248	6-table LEFT JOIN chain
Broken flow	Which sales orders have no delivery?	LEFT JOIN + NULL detection
Payment gap	Show billing documents with no payment	Multi-step LEFT JOIN
Product analytics	Which products appear in the most billing documents?	JOIN + COUNT DISTINCT
Guardrail	What is the capital of France?	Rejected — off-topic
Guardrail	Write me a poem	Rejected — off-topic

11. Running Locally

Prerequisites

- Go 1.21+
- Node.js 18+
- PostgreSQL (or Supabase account)

- Groq API key

Setup Steps

1. Clone the Repository

```
git clone https://github.com/ByteBeginner-dev/llm-powered-o2c.git
cd FDE-DodgeAI
```

2. Configure Backend

```
cd backend
nano .env
# Fill in DATABASE_URL, GroqAPIKey, DataDir, PORT
```

3. Run Backend

```
go run cmd/main.go
```

(Migrates schema, ingests data, and starts the server)

4. Run Frontend

```
cd ../frontend
npm install
npm run dev
```

5. Open in Browser

```
http://localhost:5173
```